

SESSION - 14

Custom Components



Table of Contentss

Table of Contentss **1**

Custom Components ***2***

Features of Components ***2***

Exercise 1: Create a custom component **3**

Exercise 2: Add a dialog field validation to the custom component **5**

Exercise 3: Add a base style to the custom component **12**

Exercise 4: Add a style system to the custom component **12**

Exercise 5: Add a Sling Model as the business logic **16**

Custom Components

Introduction

Adobe Experience Manager (AEM) components are used to hold, format, and render the content on your webpages. Depending on the component you want to implement in your project, you can extend or customize an existing component, rather than defining and developing the entire structure from the beginning.

Objectives

After completing this module, you will be able to:

- Explain the important features of components
- Create a custom component
- Add a dialog field validation to the custom component
- Add a base style to the custom component
- Add a style system to the custom component
- Add a Sling model as the business logic
- Add a selector to the custom component
- Create localization information
- Extend the core component
- Extend the AEM navigation UI

Features of Components

Some common component features are:

- **Component Placeholder:** Helps validate if the component is on the page and is available only in the page Edit mode. The placeholder is not available in the page Preview mode or on the publish instance. If the content is added to the component, the placeholder does not appear on the page.
- **Dialog Validation:** Provides a straightforward validation framework that helps create custom form element validators and interfaces with them programmatically. Registering custom validators is done by calling a jQuery based $.validator.register method. The register method takes a single JavaScript object literal argument. The parameter looks for four properties: selector, validate, show, and clear, of which only selector is required.
- **Style System**: Enables a template author to define style classes in the content policy of a component so that a content author can select them when editing the component on a page. These styles provide alternative visual variations of a component. This eliminates the need to develop a custom component for each style or to customize the component dialog to enable

such style functionality. It leads to more reusable components that can be quickly and easily adapted to the needs of content authors without any AEM back-end development.

- **Sling Models for Components**: When developing an AEM project, you can define a model object (a Java object) and map that object to Sling resources. A Sling Model is implemented as an OSGi bundle. A Java class located in the OSGi bundle is annotated with @Model and the adaptable class (for example, @Model(adaptables = Resource.class). The data members (Fields) use @Inject annotations. These data members map to node properties.
- **Sling Model Exporter:** Enables new annotations to be added to Sling Models that define how the Model can be exported as JSON. With Sling Model Exporter, you can obtain the same properties as a JSON response, without creating a Sling Servlet. You need to export the Sling Model by using the Jackson exporter. The Sling Model Exporter can be used as a Web service or as a REST API.
- **Selectors**: Provides a way to choose the script to be rendered when a user requests a page. During the resource resolution step, if the first character in the request URL after the resource path is a dot (.), the string after the dot up to (but not including the last dot before the slash character, or the end of the request URL) comprises the selectors. If the resource path spans the complete request URL, no selectors exist. Also, if only one dot follows the resource path before the end of the request URL or the next slash, no selectors exist.
- **i18n Translation**: Provides a Strings & Translations console for managing the various translations of texts used in the component UI. The translator tool helps manage English strings and their translations. The dictionaries are created in the repository. From this console, you can search, filter, and edit both the English and translated texts. You can also export dictionaries to XLIFF format for translating, and then import the translations back into the dictionaries. It is also possible to add the i18n dictionaries to a translation project from the console. You can either create a new dictionary or add a dictionary to an existing project.

Exercise 1: Create a custom component

You need to create a custom component, Stockplex component, to display stock information based on the user's specification. The user should be able to change the design as well as export the content as JSON. To create the Stockplex component:

1. Navigate to **Adobe Experience Manager** > **Tools** > **CRXDE Lite**. The **CRXDE Lite** page opens.
2. Navigate to the **/apps/training/components** folder.
3. Right-click the **content** folder, and click **Create** > **Create Component**. The **Create Component** dialog box opens.
4. Enter the following details in the dialog box:

   a. **Label**: **stockplex**

   b. **Title**: **StockPlex**

   c. **Description**: We.Train Complex Stock Component

   d. **Group**: **We.Train**

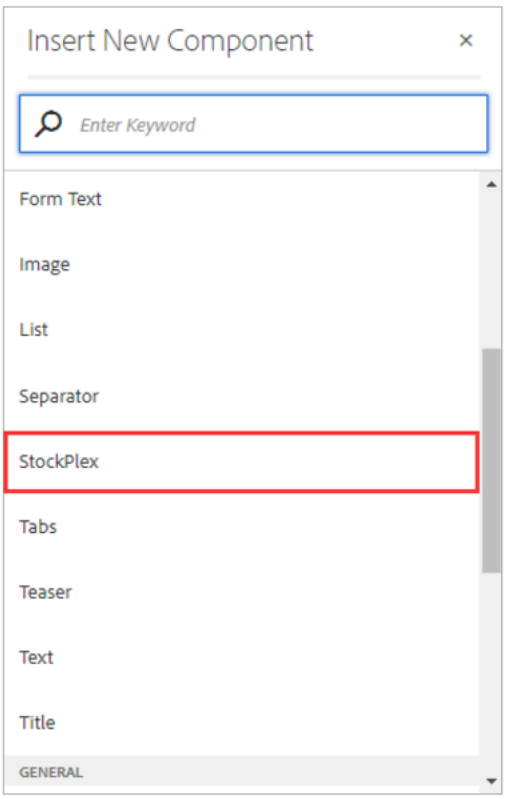
5. Click **Next**.
6. Keep the **Advanced Component** Settings as is and click OK. The component is created.
7. Click Save All from the actions bar.
8. Expand the stockplex component and double-click **stockplex.jsp**. The editor opens on the right.
9. Delete the existing content and click Save All.
10. Right-click stockplex.jsp, select Rename, and change the file name to stockplex.html. Click **Save All**.
11. Double-click **stockplex.html** to open the script for editing.
12. Replace the existing code with code from **placeholder_stockplex.html** in the exercise folder.
13. Click Save all.

You must create a cq:dialog node to enable the component delete functionality and to provide a means for the author to enter content.

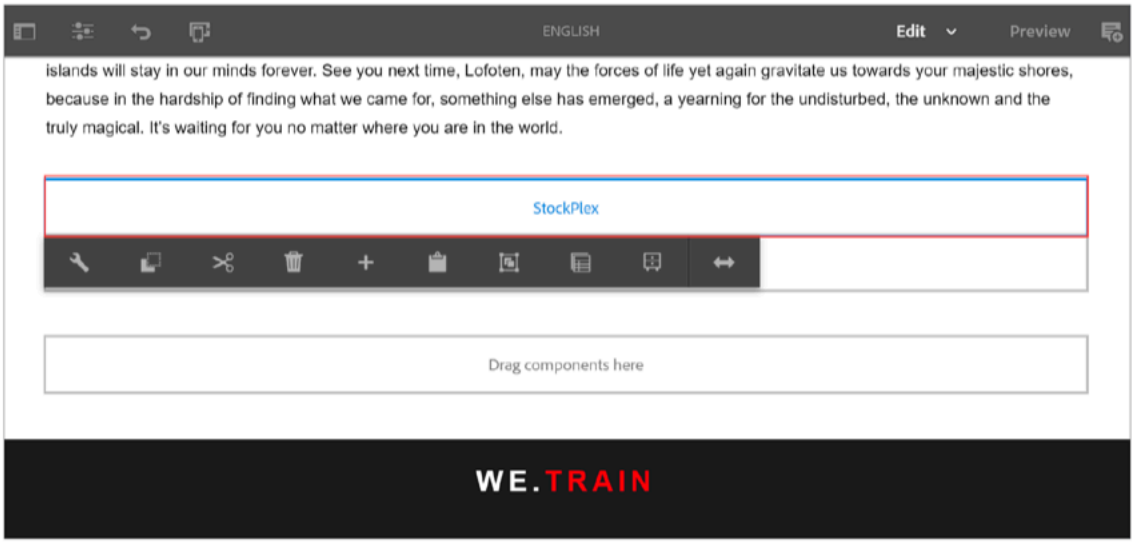
14. Right-click the stockplex component and click **Create** > **Create Node**. The Create Node dialog box opens.
15. Update the following details:

    a. Name: cq:dialog

    b. Type: nt:unstructured
16. Click OK. The node is created.
17. Click Save All.
18. Open The English page of the **We.Train** site in the Edit mode. (http://localhost:4502/editor.html/content/we-train/en.html)
19. At the bottom of the page, click the Drag components here placeholder, and click the **Insert component** icon from the component toolbar. The Insert New Component dialog box opens.
20. The Stockplex component is now available on the page, as shown, since the component was added to the We.Train group. If the Stockplex component does not appear on the page, you must enable the component in the Layout Container of the Content Page template.



21. Select the Stockplex component from the list. The stockplex component is added to the page, as shown:



In this exercise, you created a custom component and added the necessary elements for an author to be able to add it to the page.

Exercise 2: Add a dialog field validation to the custom component

In this exercise, you will build a dialog box with a validation formfield. The dialog will be based on a core component dialog, which ensures minimal customization is required. The customization includes creating a validation form field that will ensure the content entered by the author is in the correct format for the component.

1. Ensure you are in the CRXDE Lite page or click http://localhost:4502/crx/de. The CRXDE Lite page opens.
2. Navigate to the **/apps/training/components/content** folder.
3. Select the stockplex node and double-click stockplex.html to open the script for editing.
4. Update the **stockplex.html** script by replacing the existing code with new code from **dialog_stockplex.html** located in the exercise folder.
5. Open the stockplex.html to review the code you added. Check the properties of the object to see if an author can add content into the dialog.
6. Click Save All.

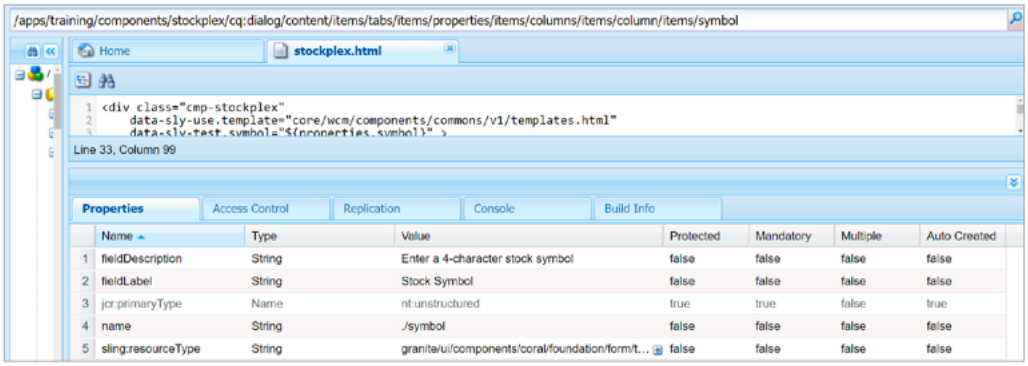
To create a dialog:

7. Select the **cq:dialog** node that is below the stockplex component.
8. Right-click the cq:dialog node and click Delete. The node is deleted.
9. Click Save All from the actions bar.
10. Navigate to the **/apps/core/wcm/components/title/v2/title/cq:dialog** node, right-click the cq:dialog node, and click Copy.
11. Navigate to the **/apps/training/components/content** folder, right-click the **stockplex** node, and click Paste.
12. On the **cq:dialog** node, update the following properties in the Properties section:

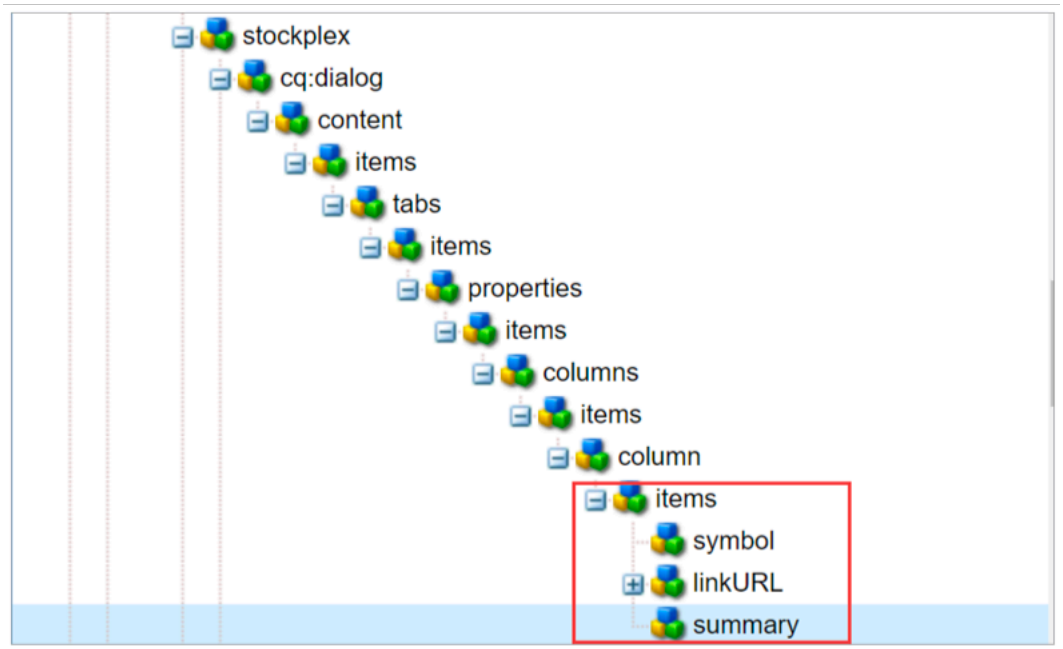
    a. Change the value of **jcr:title** to Stockplex.

    b. Right-click the helpPath property and click Delete to delete it.
13. Click Save All from the actions bar.
14. Navigate to the **/apps/training/components/content/stockplex/cq:dialog/content/items/tabs/items/properties/items/columns/items/column/items** node.
15. Right-click the types node and click Delete to delete it.
16. Click Save All from the actions bar.
17. Right-click the **defaulttypes** node and click Delete to delete it.
18. Navigate to the **/apps/training/components/content/stockplex/cq:dialog/content/items/tabs/items/properties/items/columns/items/column/items/title** node.
19. Right-click the title node, click Rename, and rename it as symbol.
20. Update the following properties for the symbol node, as shown:

Name	Type	Value
name	String	./symbol
fieldLabel	String	Stock Symbol
fieldDescription	String	Enter a 4-character stock symbol



21. Right-click the symbol node, and click Copy.
22. Right-click the items node that is above the symbol node, and click Paste.
23. Rename the Copy of symbol node to summary. The summary and symbol nodes are siblings as shown:



24. Update the following properties of the summary node:

Name	Type	Value
name	String	./summary
fieldLabel	String	Summary of Stock
fieldDescription	String	Enter a summary description of the stock

25. Right-click the symbol node, and click Copy to copy it.
26. Right-click the items node that is above the symbol node, and click Paste to paste the node.
27. Rename the Copy of symbol node to stockdetails.
28. Update the following properties of the stockdetails node:

Name	Type	Value
name	String	./showStockDetails
fieldDescription	String	Include requestDate, upDown, openPrice, range high/low, volume, and others
sling:resourceType	String	granite/ui/components/coral/foundation/form/checkbox

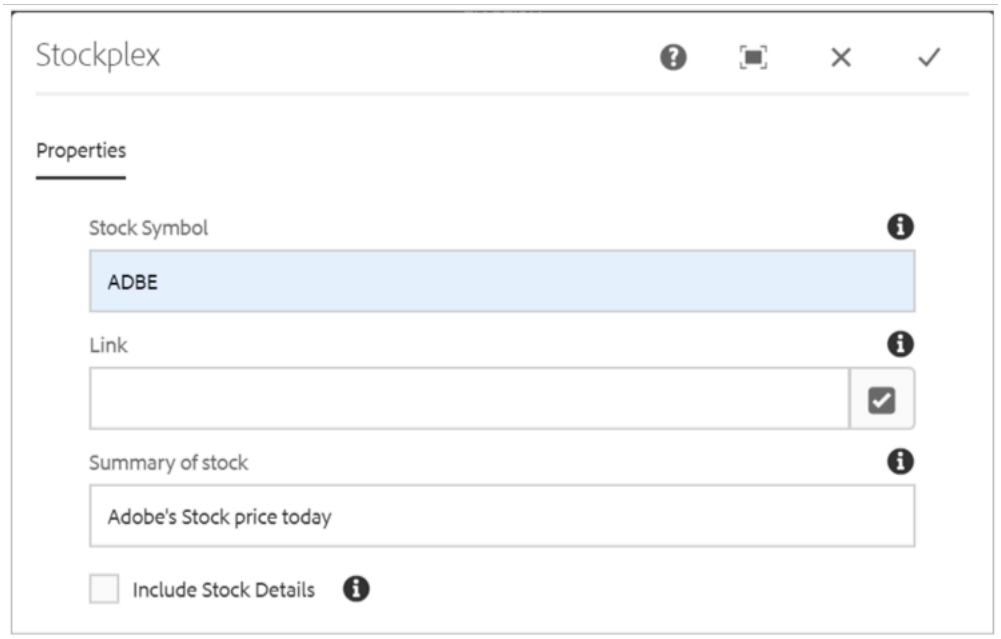
29. Add the following new properties to the stockdetails node:

Name	Type	Value
text	String	Include Stock Details
value	Boolean	true

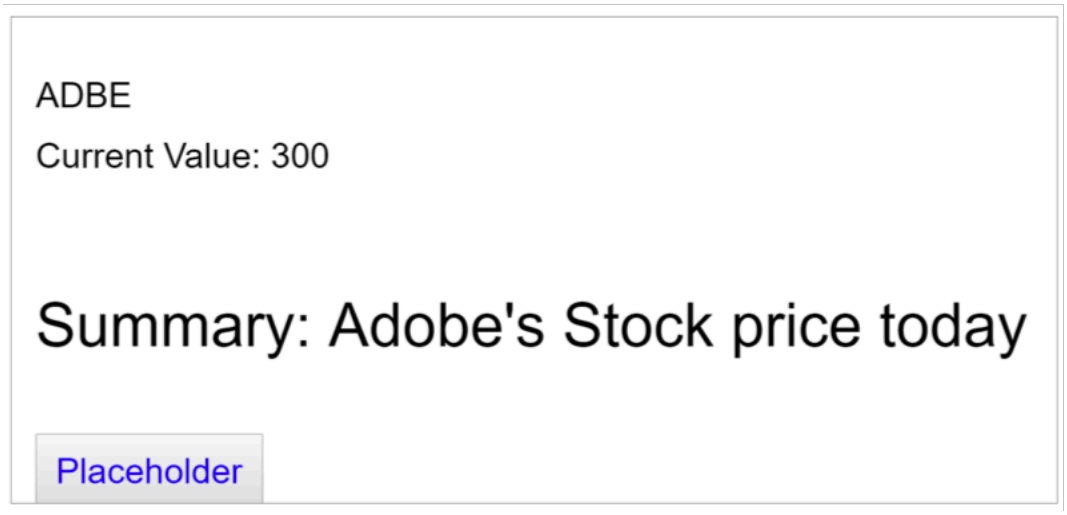
30. Select the fieldLabel property, right-click it, and click Delete from the list.
31. Click Save All from the actions bar.
32. Navigate to the tab where the English page is open or click http://localhost:4502/editor.html/content/we-train/en.html
33. Select the stockplex component you added in a previous task, and click the Configure (wrench) icon. The Stockplex dialog box opens.
34. Provide the following values, as shown:

    a. Stock Symbol: ADBE

    b. Summary of Stock: Adobe's Stock price today

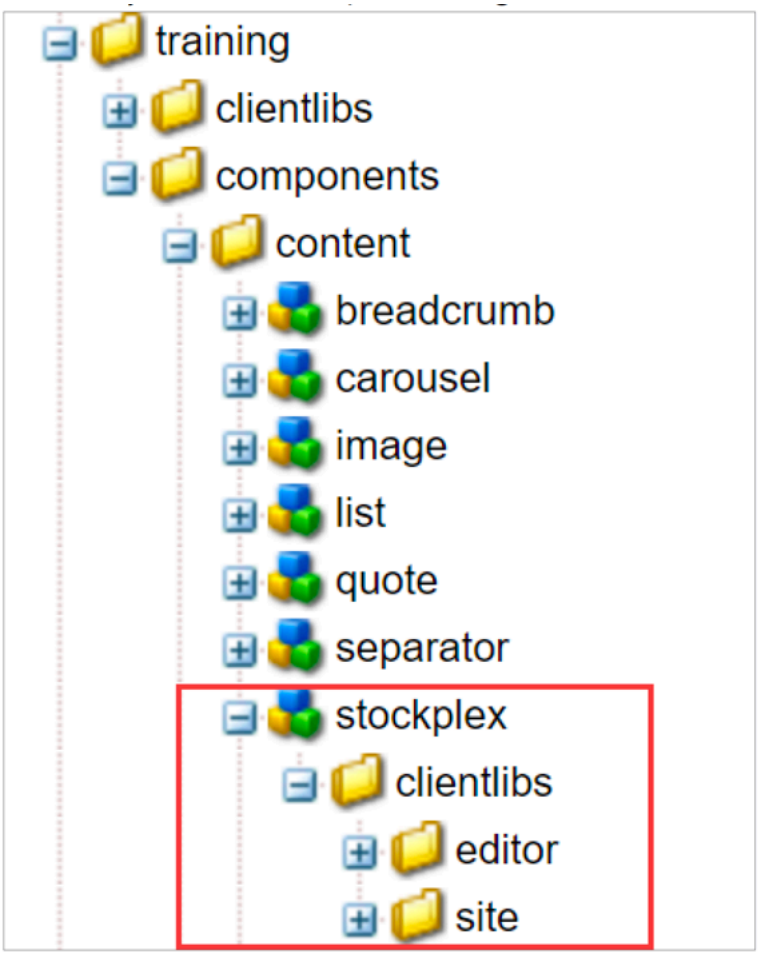


35. Click **Done** (checkmark). You can view the content rendered on the stockplex component in the English page.



Now that you have a basic dialog with three form fields, you need to create validation of the stock symbol. As the symbol description suggests, the user must enter a four letter symbol. You can accomplish this by including client-side JavaScript with a client library.

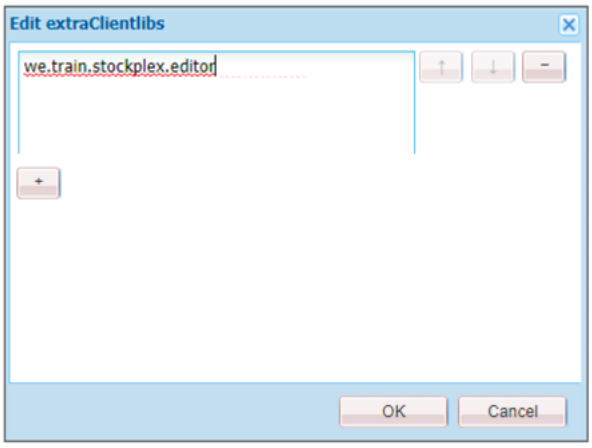
36. Open package manager and upload **stockplex-clientlibs-all.zip** package from the exercise folder.
37. Install the package.
38. Now from CRXDE Navigate to the **/apps/training/components/content/stockplex/clientlibs** folder. Notice that the content package contains two client libraries for the stockplex component, as shown:
    - **editor:** A client library containing a validation script for a dialog.
    - **site:** A css/js file for the component design.



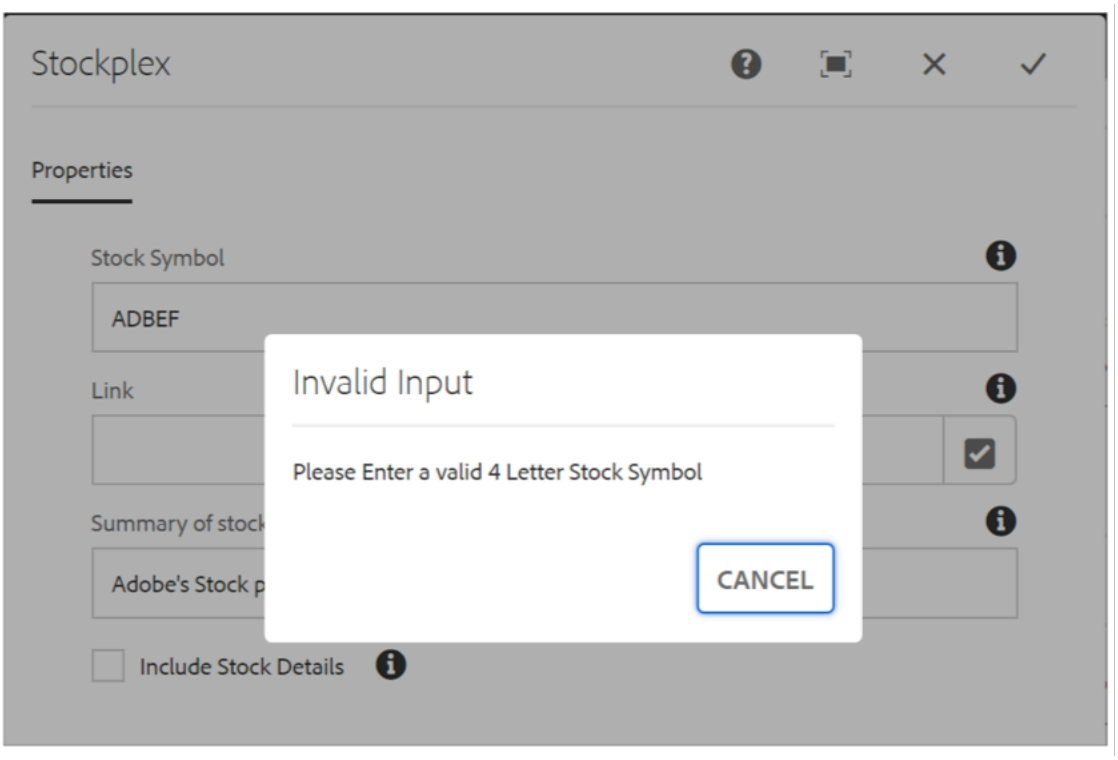
To use the stockplex editor client library, you must add it to the dialog box. If you want to define styling and behavior for your component, you can create a dedicated client library that defines your custom CSS/LESS and JS.

To have your client library loaded solely for your component dialog (that is, it will not be loaded for another component), you must set the property extraClientLibs ** of your dialog to the category name of the client library you just created. This is recommended if your client library is quite large and/or your field is specific to that dialog and will not be needed in other dialogs.

39. Navigate to the **/apps/training/components/content/stockplex/cq:dialog** node.
40. In **Properties** section, double-click in the **extraClientlibs**. The Edit **extraClientlibs** dialog box opens.
41. Update the value to **we.train.stockplex.editor**, as shown, and click OK. The existing value is replaced with the new value.



42. Click Save All from the actions bar.
43. Navigate to the tab where the English page is open.
44. Select the stockplex component, and click the Configure (wrench) icon. The Stockplex dialog box opens.
45. In the Stock Symbol field, type more than four characters, and then click the Done (checkmark) icon. The Invalid Input dialog box opens with a message, as shown. This indicates that you created an author dialog box for a custom component and added JavaScript in a client library to perform validation on one of the fields.



46. Click Cancel to close the dialog box, and then click the Cancel (X) icon in the **Stockplex** dialog box to discard your changes.

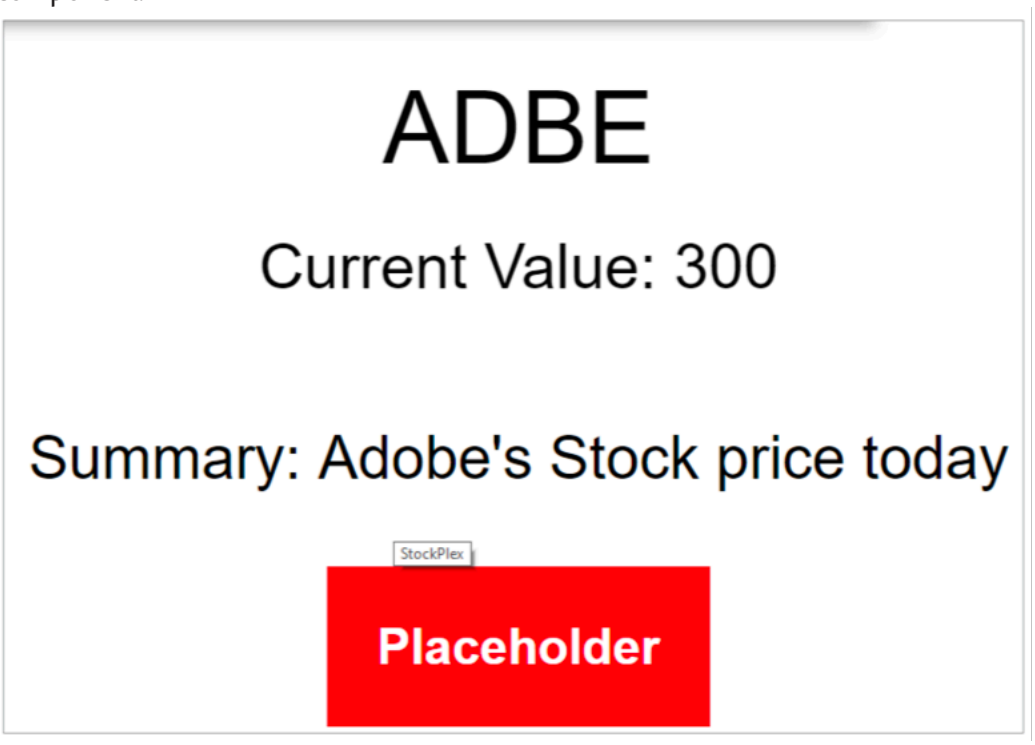
Exercise 3: Add a base style to the custom component

In the previous exercise, you installed the client libraries for the stockplex component. In this exercise, you will add a design to the stockplex component.

1. Ensure you are in CRXDE Lite or click http://localhost:4502/crx/de. The CRXDE Lite page opens.
2. Navigate to the **/apps/training/components/content/stockplex/clientlibs/site** node. Notice that the clientlibs for the design of the **stockplex** component are available in this folder.

To use the design in stockplex component, you must add the design to the HTL.

3. Navigate to the /apps/training/components/content folder.
4. Select the stockplex node and double-click the stockplex.html to open the script for editing.
5. Update the stockplex.html script by replacing the existing code with the new code in **clientlibs_stockplex.html** located in the exercise folder.
6. Click **Save All**.
7. Navigate to the tab where the English page is open, and refresh the page. Notice that the stockplex component has a new design, as shown. You modified your component's script to use the CSS styles from the component's client library. After modifying, you can see the change in the appearance of the component.



Exercise 4: Add a style system to the custom component

In this exercise, you will provide content authors the ability to change the design of a component in the authoring environment without having to write the code. The selectable designs must be pre-created by the design team to help authors choose different designs they want to use on the page.

1. Ensure you are in CRXDE Lite or click http://localhost:4502/crx/de. The CRXDE Lite page opens.

To view different designs that you want the authors to access:

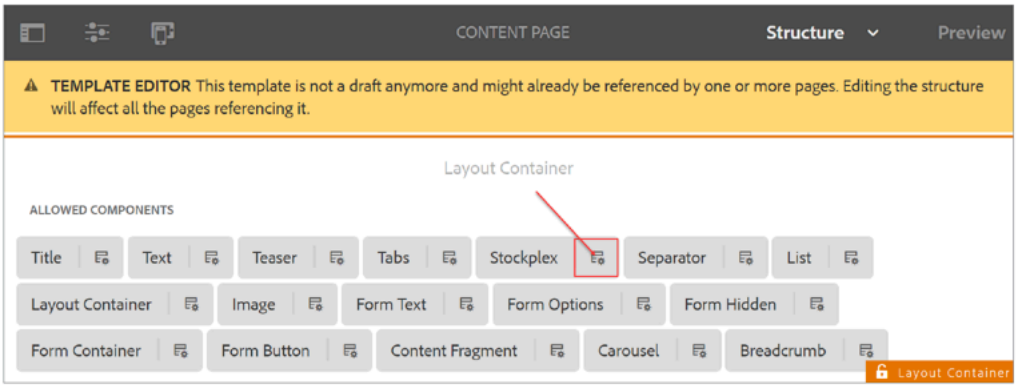
2. Navigate to the **/apps/training/components/content/stockplex/clientlibs/site/css/ stockplex.css** node.
3. Double-click **stockplex.css** and notice **cmp-stockplex--italic**, **cmp-stockplex--bold**, and **cmp-stockplex--red** in the editor. These classes are not used in the base stockplex code and must be added to the style system.

To help a template author define styles in a content policy, you must add the style system as a cq:design_dialog. Just like cq:dialogs, you will copy a cq:design_dialog from a core component and customize it.

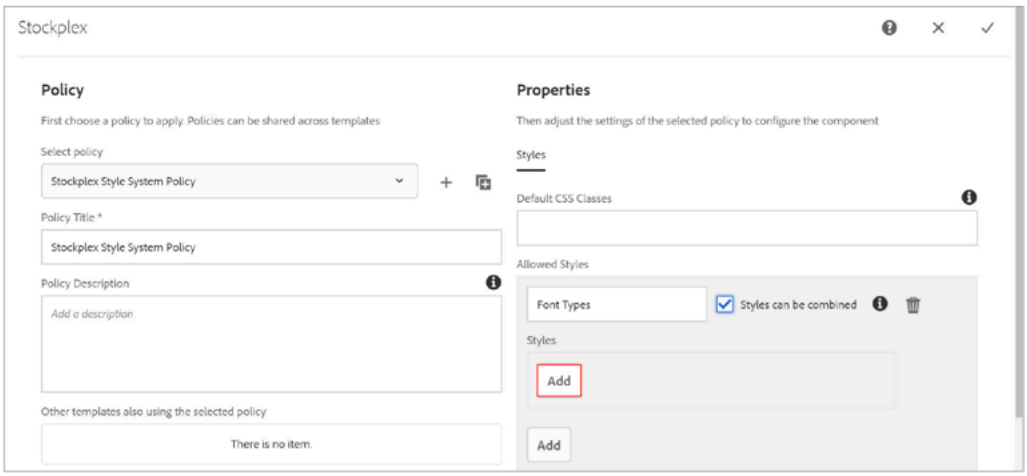
4. Navigate to the **/apps/core/wcm/components/text/v2/text node**, right-click the **cq:design_dialog** node, and click **Copy** to copy the node.
5. Navigate to the **/apps/training/components/content/stockplex** node, and click Paste to paste the node. The node is added to the stockplex component.
6. Click **Save All** from the actions bar.
7. Select the **cq:design_dialog** node, and on the Properties tab, update the **jcr:title** property to **Stockplex**.
8. Click Save All from the actions bar.
9. Navigate to the **/apps/training/components/content/stockplex/cq:design_dialog/content/items/tabs/items** node, right-click the plugins node, and click Delete. The node is deleted.
10. Click Save All from the actions bar.

Now, as a template author, you will add the style system to the **stockplex** content policy.

11. Click CRXDE Lite from the header bar. The **Navigation** page opens.
12. Click the **Tools** (hammer) icon. The Tools console opens.
13. Ensure you are in the **General** section and click the Templates tile. The Templates console opens.
14. Click the **We.Train** folder, select the **Content Page** template, and click Edit (e) from the actions bar. The **Content Page** template editor opens on a new tab.
15. 15. In the **Layout Container** component, click the **Policy** icon next to the **Stockplex** component, as shown. The **Stockplex** policy wizard opens.

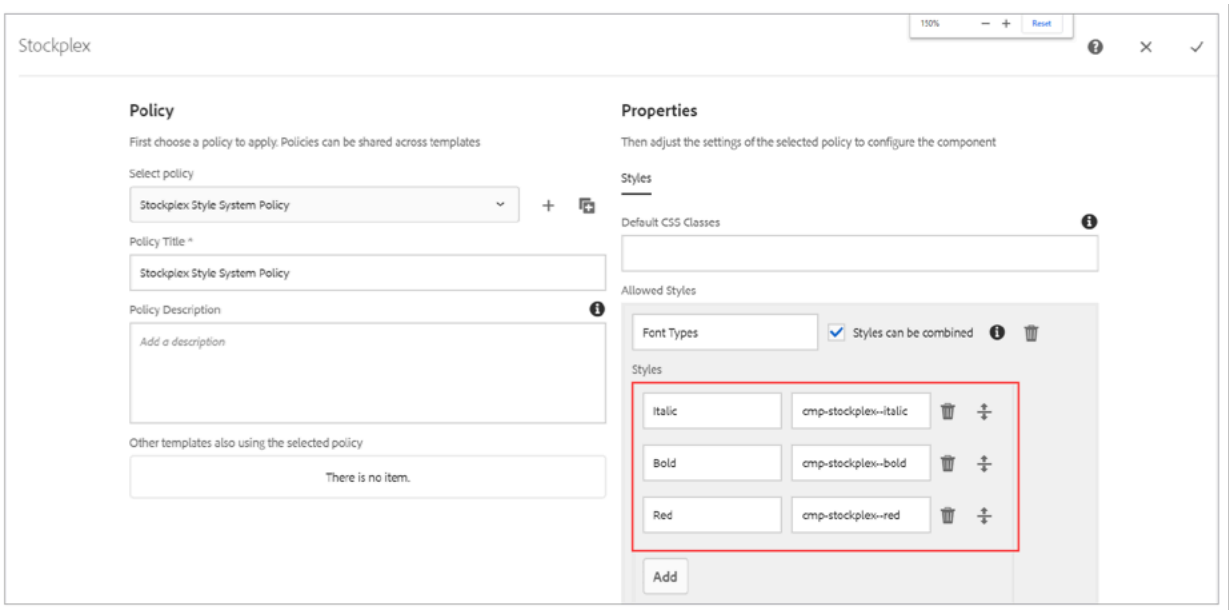


16. In the **Policy** section, type **Stockplex Style System** Policy in the Policy Title field.
17. In the **Properties** section, under the Styles tab, click the Add button below the Allowed Styles field.
18. In the **Group Name** field, type **Font Types**, and select the **Styles can be combined** checkbox.
19. Click the **Add** button that is below the **Styles** field, as shown. A new style field is created.



20. Enter the **Style Name** as **Italic** and **cmp-stockplex--italic** as the CSS Classes.
21. Click the **Add** button below the **Italic** style you just added. A new style field is created.
22. Enter the **Style Name** as **Bold** and **cmp-stockplex--bold** as the **CSS Classes**.
23. Click the **Add** button again, and enter the **Style Name** as **Red** and **cmp-stockplex--red** as the **CSS Classes**.

Your styles should look similar to the one shown in the following screenshot:

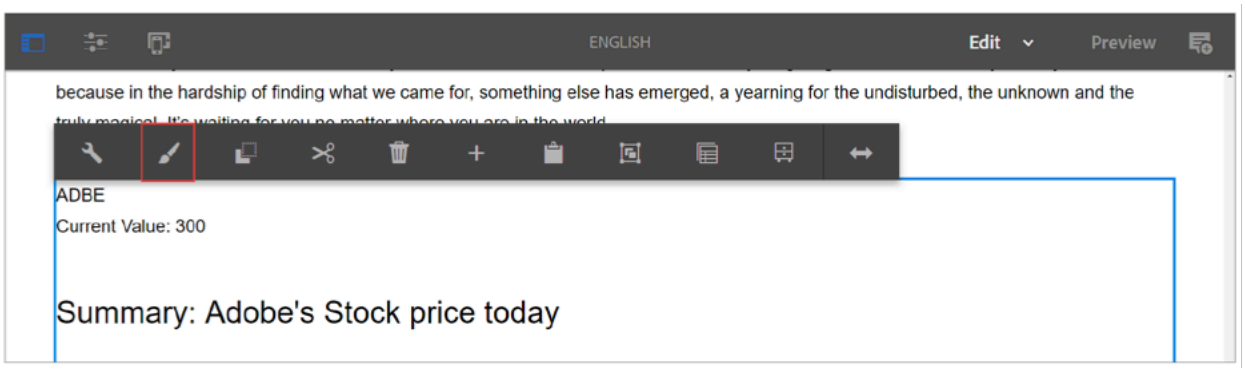


24. Click the **Done** (checkmark) icon in the Stockplex wizard. The **Content Page** template editor opens.

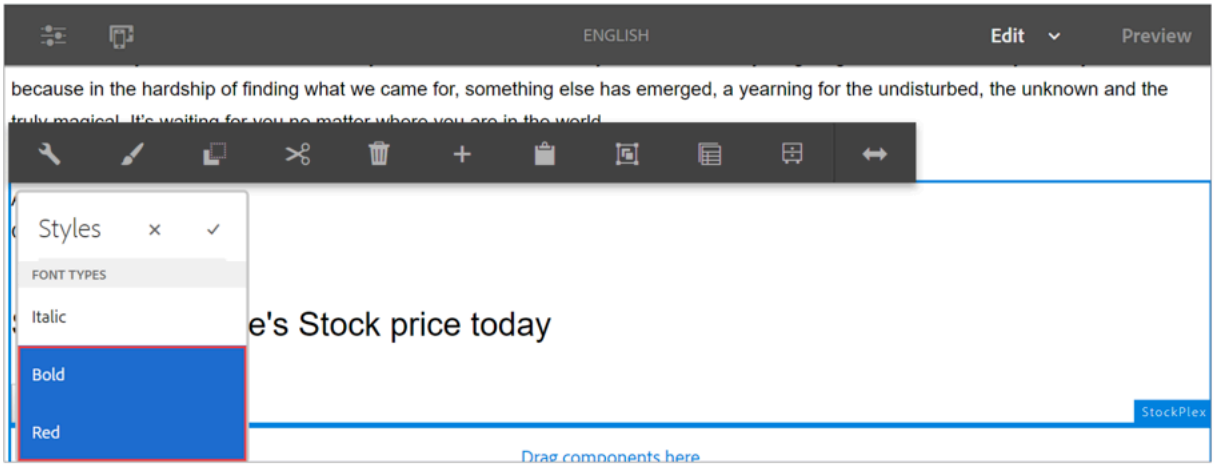
Now that you have the CSS developed by the designer, and the style system is added by the template editor, authors can use the style system.

25. Navigate to the **English** page of the We.Train site, and refresh it.

26. Select the **Stockplex** component. You should now see the **Styles** (paintbrush) icon in the component toolbar, as shown:



27. Click the **Styles** (paintbrush) icon from the actions bar, and click **Italic**, **Bold**, or **Red**, and observe the changes. You have now modified your component's script to use the CSS styles from the component's client library. After modifying, you noticed changes in the appearance of the component. If you are unable to view any changes to the styles in the stockplex component, refresh the English page.



Exercise 5: Add a Sling Model as the business logic

In this exercise, you will use a Sling model as the business logic of the stockplex component. This will allow the component to be used for both HTML rendering and JSON output.

1. The Sling Model you will install is called Stockplex.java and is compiled into an OSGi bundle. Observe the raw java code in your exercise files.
2. To do that install **training.core-0.0.1-SNAPSHOT.jar** bundle(provided in exercise folder) from felix console.

To use the Stockplex model, you must update the stockplex.html script:

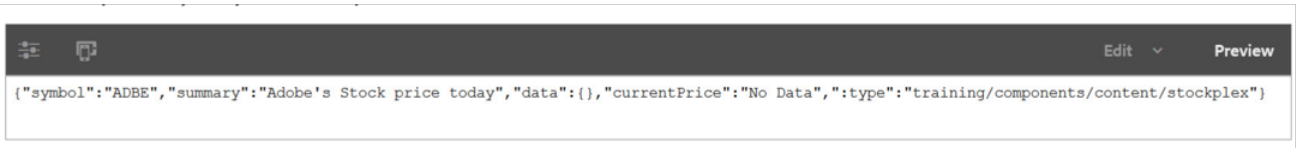
3. Ensure you are in CRXDE Lite or click http://localhost:4502/crx/de. The CRXDE Lite page opens.
4. Navigate to the **/apps/training/components/content** node.
5. Select the **stockplex** node, and double-click **stockplex.html** to open the script for editing.
6. Update the **stockplex.html** script by replacing the existing code with the new code in **slingmodel_stockplex.html** from the exercise files provided to you.
7. Click **Save All** from the actions bar. Review the code you added. Notice that the data-sly-use="com.adobe.training.core.models.Stockplex" and notice how all values for the content are coming from the model rather than the dummy data. In addition, notice how the button is now been updated with an href="${resource.path @ selectors='model', extension='json'}". When you request a resource with model.json, if there is a sling model for that resource, it will display the JSON output.

To observe the JSON output:

8. Navigate to the **English** page or click http://localhost:4502/editor.html/content/we-train/en.html. The **English** page of We.Train site opens.
9. Refresh the page. Notice how you no longer see the dummy data, and the button's text is updated to **See this stock via JSON**, as shown:



10. To view the output:
11. Click **Preview** from the page toolbar and click the **See this stock via JSON** button on the page. The page with the following output opens on the same tab, as shown. You modified your component to receive data from a java Sling Model and exposed the Sling Model's JSON Export capability in your component's interface.



12. Additionally, you can view the stockplex JSON apart of the entire page's JSON output. Click **Page Information > View as Published**. From the URL, remove **.html?wcmmode=disabled** and add **.model.json**. This is the JSON output of the entire page including the stockplex sling model.